



构建网络



构建网络

- 构建网络是深度学习最重要的一个步骤。
- 若网络太简单则无法学习到足够丰富的特征，若网络太复杂则容易过拟合。
- 而且，对不同的数据，要选择合适的网络结构才能取得较好的结果。

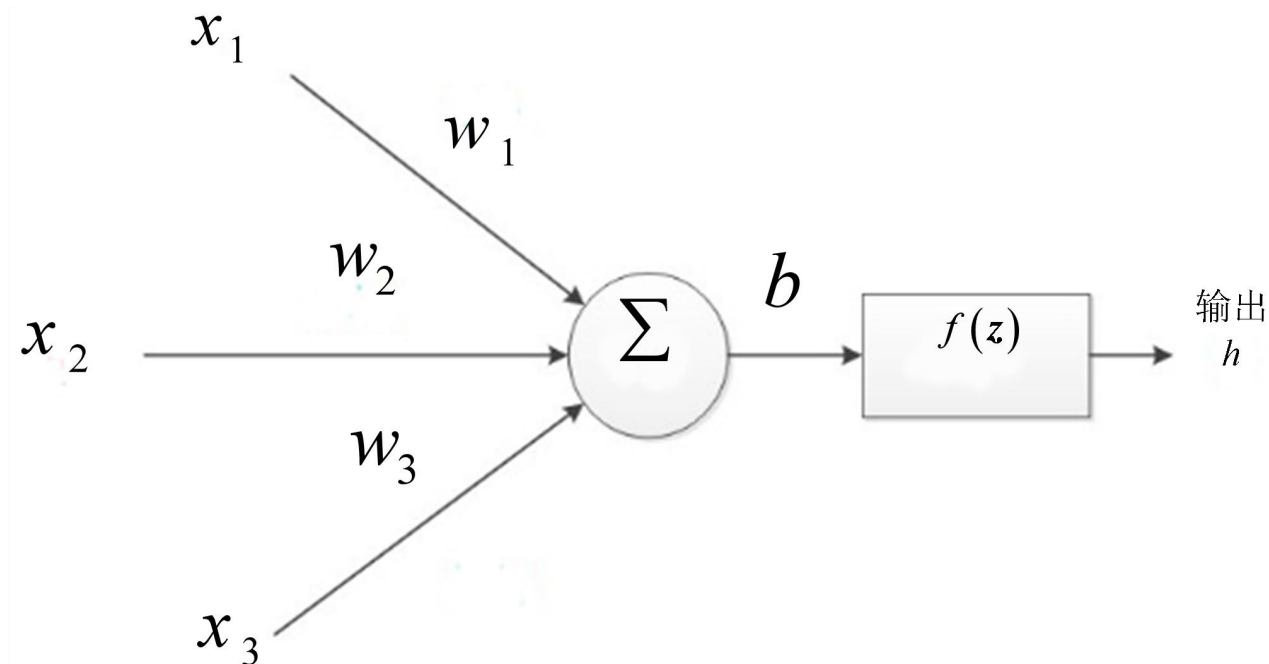


常用的网络构建方法

- 在深度学习中，构建网络通常是搭建一个完整的神经网络结构。
- 神经网络（Neural Networks）是一种应用类似于大脑神经突触连接的结构进行信息处理的算法。
- 神经网络已经被用于解决分类、回归等问题，同时被运用在机器视觉、语音识别等应用领域上。
- 神经网络是由具有适应性的简单单元组成的广泛并行互连网络，它的结构能够模拟生物神经系统对真实世界的交互反应。

常用的网络构建方法

- 一个简单的神经元结构如图所示。

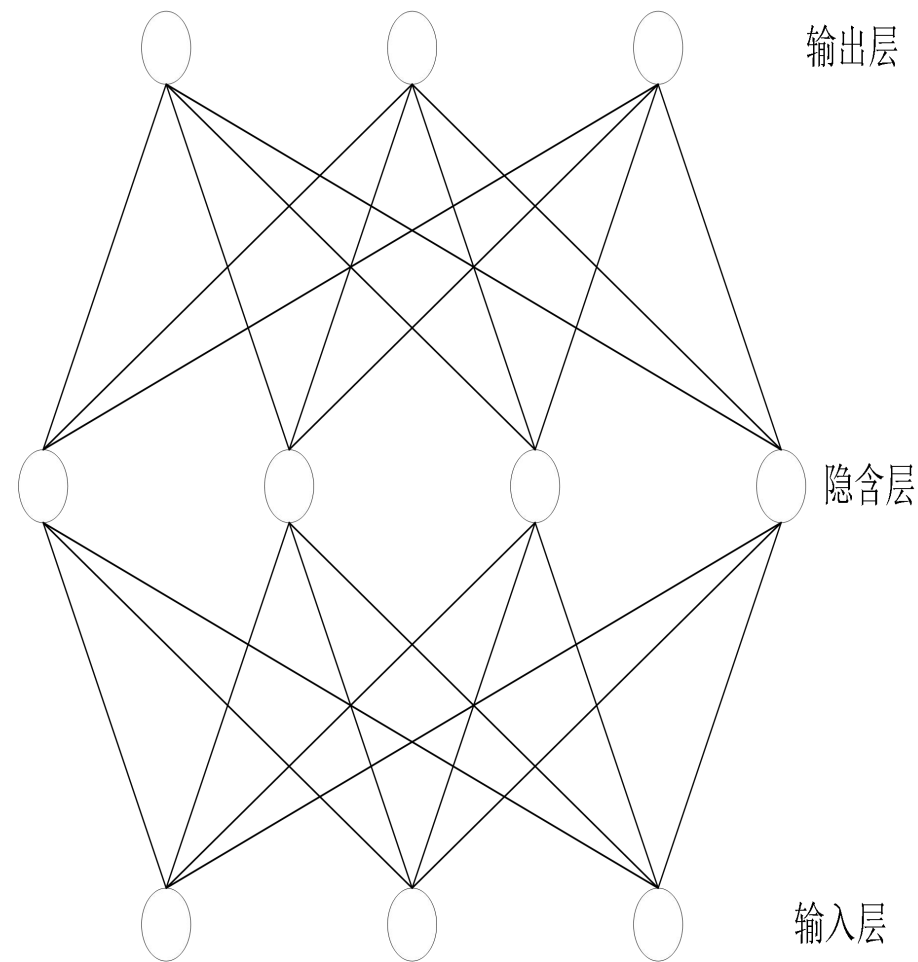


- 在图中， w_1 、 w_2 和 w_3 表示权重， b 表示偏置项， $f(z)$ 表示激活函数， x_1 、 x_2 和 x_3 是神经元的输入值，输出值

$$h_{w,b}(\mathbf{x}) = f(\mathbf{w}^T \mathbf{x}) = f\left(\sum_{i=1}^3 w_i x_i + b\right)$$

常用的网络构建方法

- 将多个神经元按一定的层次结构连接起来，即可得到一个神经网络。
- 使用神经网络需要确定网络连接的拓扑结构、神经元的特征和学习规则等。
- 常见的神经网络的层级结构如图所示，每层神经元与下一层的神经元全部互连，同层之间神经元不存在连接关系。
- 图为简单的全连接神经网络，其中输入层神经元接收信号，最终输出结果由输出层神经元输出。



常用的网络构建方法

- 隐含层是指除了输入、输出层以外的其他层，是模型的黑箱部分，通常可解释性较差。
- 值得注意的是，如果单个隐含层网络不能满足实际生产需求，那么可在网络中设置多个隐含层。
- 深度学习神经网络中的常见的网络层有卷积层、池化层、正则化层、归一化层和全连接层。

常用的网络构建方法

- 网络层的先后顺序通常是，卷积层优先构造，池化层放置在卷积层之后，正则化层和归一化层放置在整個网络中间偏后的位置，全连接层放置在网络的后端或多个卷积层后。
- 在PyTorch框架中，常用的构建网络的方法有继承Module类和使用Sequential容器。

继承Module类构建网络

- Module类是PyTorch框架提供的一个基类，在搭建神经网络时候可以通过继承Module类，使得搭建网络的过程变得简单。
- 继承Module类构建网络，其中__init__()方法初始化了网络中需要使用的网络层，完全相同的网络层仅可被初始化一次，但可被多次进行调用，forward()方法用于设置网络中数据的传播，并返回网络的输出。

继承Module类构建网络

```
import torch
from torchsummary import summary
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
class model1(torch.nn.Module):
    def __init__(self):
        super(model1, self).__init__()
        self.linear1 = torch.nn.Linear(3, 5)
        self.linear2 = torch.nn.Linear(5, 2)
```

```
    def forward(self, x):
        x = self.linear1(x)
        x = self.linear2(x)
        return x
```

```
model = model1().to(device)
summary(model, (8, 3))
```

```
-----
              Layer (type)              Output Shape          Param #
=====
              Linear-1                  [-1, 8, 5]              20
              Linear-2                  [-1, 8, 2]              12
=====
Total params: 32
Trainable params: 32
Non-trainable params: 0
-----
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.00
Estimated Total Size (MB): 0.00
-----
```

继承Module类构建网络

- 从Layer列中可以查看所构建网络的结构，从Output Shape列中可以查看每层网络输出数据的维度。
- 从Param列中可以查看每层网络的参数个数。

使用Sequential容器构建网络

- Sequential是一个有序的容器，网络层将按照在传入构造器的顺序依次被添加到计算图中执行，使用Sequential容器构建网络。
- 同时以神经网络模块为元素的有序字典也可以作为Sequential容器的传入参数。

```
import torch
from torchsummary import summary
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
model2 = torch.nn.Sequential(torch.nn.Linear(3, 5),
                              torch.nn.Linear(5, 2))
```

```
model = model2.to(device)
summary(model, (18, 3))
```

Layer (type)	Output Shape	Param #
Linear-1	[-1, 18, 5]	20
Linear-2	[-1, 18, 2]	12

Total params: 32

Trainable params: 32

Non-trainable params: 0

Input size (MB): 0.00

Forward/backward pass size (MB): 0.00

Params size (MB): 0.00

Estimated Total Size (MB): 0.00



激活函数

- 在神经网络中，全连接层只是对上层输入进行线性变换，而多个线性变换的叠加仍然是一个线性变换，即含有多个全连接层的神经网络与仅含输出层的单层神经网络等价。
- 解决问题的一个方法是，在全连接层后增加一个非线性的变换，将非线性变换后的结果作为下一个全连接层的输入。
- 这个非线性函数被称为激活函数（Activation Function），神经网络可以逼近任意函数的能力与激活函数的使用不可分割。
- 常见的激活函数有ReLU、Sigmoid、Tanh和Softmax等。

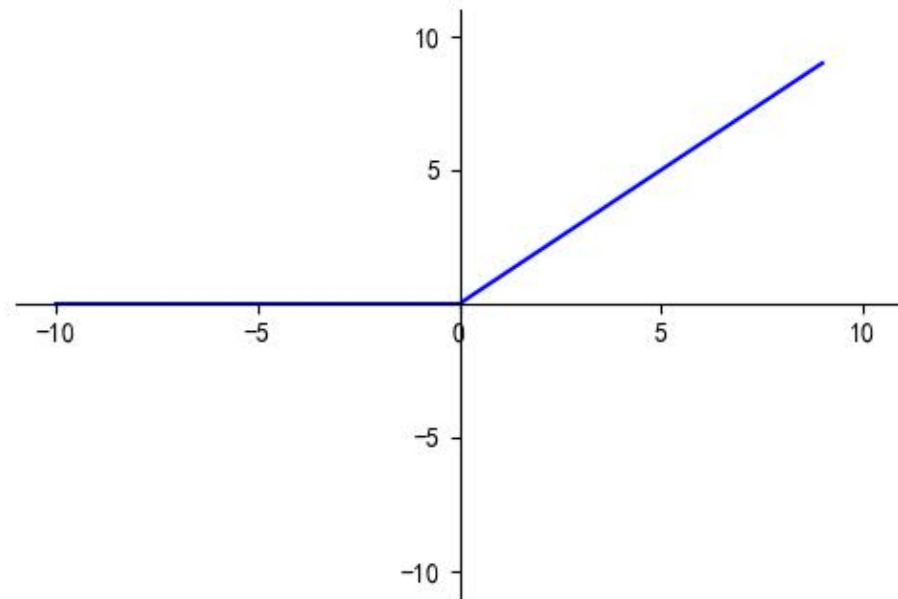
1. ReLU函数

- 线性整流函数（Rectified Linear Unit, ReLU），又称修正线性单元，是一种人工神经网络中常用的激活函数，通常指代以斜坡函数及其变种为代表的非线性函数。
- ReLU函数的表达式如式所示，其中 x 为神经元的输入值。

$$f(x) = \begin{cases} \max(0, x), & x \geq 0 \\ 0, & x \leq 0 \end{cases}$$

激活函数

- ReLU函数的几何表达如图所示。



- 由图可知，当输入为正时，ReLU函数的输入与输出均始终保持线性关系，当输入趋于正无穷时，输出也趋于正无穷；当输入为负值时，输出为0。

➤ ReLU函数的优点如下。

- 反向传播时，可以避免梯度消失。
- 使一部分神经元的输出为0，形成稀疏网络，减少了参数的相互依存关系，缓解了过拟合问题的发生。
- 求导简单，整个过程的计算量节省很多。

➤ ReLU的缺点如下。

- 左侧神经元为0，导致部分神经元死亡，不再更新。
- 输出非负，所有参数的更新方向都相同，可能导致梯度下降时出现震荡。

激活函数

- relu类的语法格式如下。

torch.nn.functional.relu(input, inplace=False)

- relu类的常用参数说明如表所示。

参数名称	说明
input	接收tensor数据，表示输入值，无默认值
inplace	接收bool，表示计算softmax的维度数，默认值为False

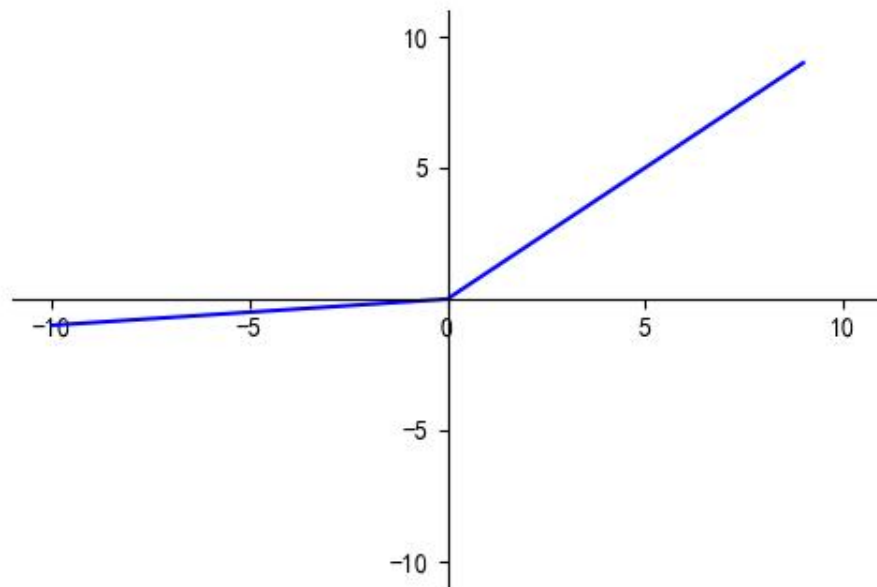
2. Leaky ReLU函数

- Leaky ReLU激活函数是ReLU激活函数的变式，主要是为了修复ReLU激活函数中负轴为0导致的问题。
- Leaky ReLU激活函数的负轴保留了非常小的常数，使得输入信息小于0时，信息得到保留。
- Leaky ReLU函数的表达式如式所示，其中 x 为神经元的输入值， α 为非负的小数。

$$f(x) = \begin{cases} \max(0, x), & x \geq 0 \\ \alpha x, & x \leq 0 \end{cases}$$

激活函数

- Leaky ReLU函数的几何表达如图所示。



- leaky_relu类的语法格式如下，leaky_relu类的常用参数说明与torch.nn.functional.relu类一致。

torch.nn.functional.leaky_relu(input, negative_slope=0.01, inplace=False)

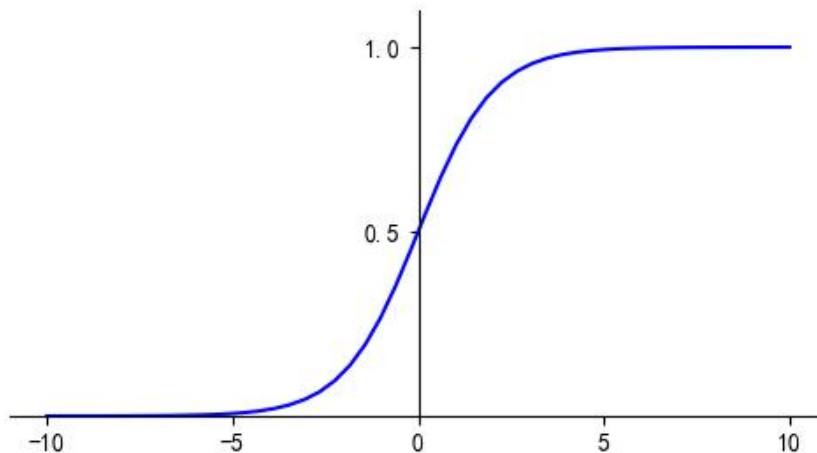
3. Sigmoid函数

- Sigmoid函数是一个在生物学中常见的S型函数，也称为S型生长曲线。
- 在信息科学中，由于其单增以及反函数单增等性质，Sigmoid函数常被用作神经网络的阈值函数，将变量映射到0到1之间。
- Sigmoid函数的表达式如式所示，其中x为神经元的输入值。

$$f(x) = \frac{1}{1 + e^{-x}}$$

激活函数

- Sigmoid函数的几何表达如图所示。



- 由图可知，纵坐标的范围在0~1的区间内，随着横坐标值从左往右增大，曲线的纵坐标值从0无限趋近于1，表示Sigmoid函数的输出范围是0到1，即对每个神经元的输出进行了归一化。
- 由于概率的取值范围是0到1，因此Sigmoid函数非常适合用在以预测概率作为输出的模型中。

➤ Sigmoid优点如下。

- Sigmoid的取值范围为(0, 1), 而且是单调递增, 比较容易优化。
- Sigmoid求导比较容易, 可以直接推导得出。

➤ Sigmoid缺点如下。

- Sigmoid函数收敛比较缓慢。
- 由于Sigmoid是软饱和, 容易产生梯度消失, 对于深度网络训练不太适合, 由上页图可知, 当 x 趋于无穷大时, 会使导数趋于0。

➤ sigmoid类的语法格式如下, 其中参数“input”表示输入值。

torch.nn.functional.sigmoid(input)

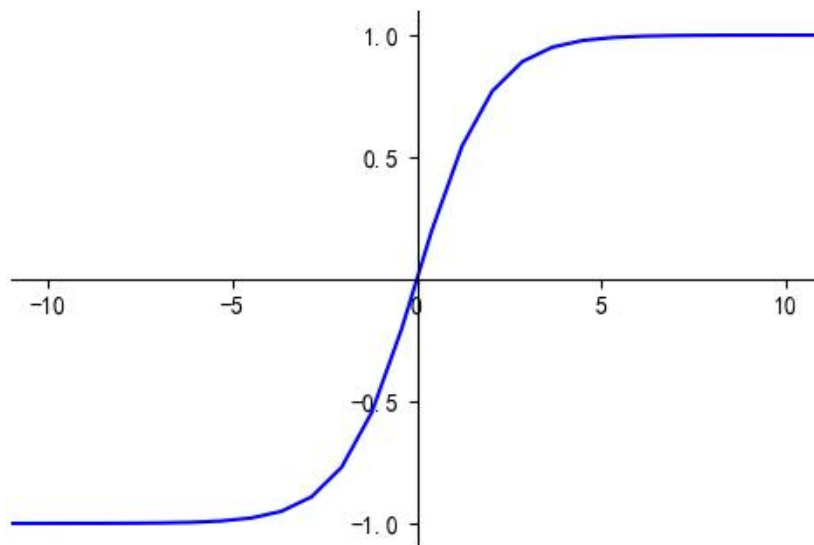
4. Tanh函数

- Tanh函数是双曲正切激活函数。
- Tanh函数的表达式如式所示，其中 x 为神经元的输入值。

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

激活函数

➤ Tanh函数的几何表达如图所示。



➤ 由图可知，当横坐标值趋于负无穷时，纵坐标值无限趋近于-1。

- 当横坐标值趋于正无穷时，纵坐标值无限趋近于1。
- 当输入的绝对值大于5时，输出几乎是平滑的并且梯度较小，不利于权重更新。

- Tanh函数跟Sigmoid函数的区别在于输出间隔，Tanh的输出间隔为2，并且整个函数以0为中心。
- 在一般的二元分类问题中，Tanh函数常用于隐藏层，而Sigmoid函数常用于输出层。
- 但这并不是固定的，需要根据特定问题进行调整。

➤ Tanh优点如下。

- 函数输出以（0,0）为中心。
- 收敛速度相对于Sigmoid更快。

➤ Tanh缺点如下。

- Tanh并没有解决Sigmoid梯度消失的问题。
- 含有较多的幂运算，增加计算所需的时间。

➤ tanh类的语法格式如下，其中参数“input”表示输入值。

torch.nn.functional.tanh(input)

5. Softmax函数

- Softmax函数常在神经网络输出层充当激活函数，将输出层的值映射到(0,1)区间，将神经元输出构造成概率分布，用于多分类问题中，Softmax激活函数映射值越大，则真实类别可能性越大。
- Softmax函数的表达式如式所示，其中i为输入神经元的第i个元素。

$$S_i = \frac{e^i}{\sum_{j=1}^n e^j}$$

- 由式可知，Softmax函数为指数形式的函数，且映射后的数值受映射前所有数值的

➤ Softmax优点如下。

- 指数函数求导的时候比较方便，加快模型的训练。
- 能够将输出值拉开距离，使不同类间的区别更明显。

➤ Softmax缺点如下。

- 通过Softmax函数计算得到的数值有可能会变得过大从而导致溢出。
- 输入均为负时，可能会出现梯度为0。

激活函数

- softmax类的语法格式如下。

torch.nn.functional.softmax(input, dim=None, _stacklevel=3, dtype=None)

- softmax类的常用参数及其说明如表所示。

参数名称	说明
input	接收tensor数据，表示输入值，无默认值
dim	接收int，表示计算softmax的维度数，无默认值
dtype	接收str，表示所需的数据类型的返回张量，无默认值

目录



构建基于CNN的猫狗分类网络

- 搭建3种不同的CNN网络进行测试，首先搭建一个经典CNN网络，包含两个卷积层、两个池化层、一个全连接层和一个丢弃层。
- 丢弃层的存在是为了防止过拟合，可以视情况保留或删除。

```
import torch
from torch import optim
import torch.nn as nn
from torch.autograd import Variable
from torchvision import transforms
from torch.utils.data import Dataset,
DataLoader
from PIL import Image
import torch.nn.functional as F
```

经典的CNN网络

构建基于CNN的猫狗分类网络

```
class CNN(nn.Module):  
    def __init__(self, num_classes=2):  
        super(CNN, self).__init__()  
        self.conv1 = nn.Conv2d(3, 16, 3,  
padding=1)  
        self.pool = nn.MaxPool2d(2, 2)  
        self.conv2 = nn.Conv2d(16, 16, 3,  
padding=1)  
        self.pool = nn.MaxPool2d(2, 2)  
        self.output = nn.Linear(16 * 64 * 64, 2)  
        self.dp1 = nn.Dropout(p=0.5)
```

```
def forward(self, x):  
    x = self.pool(F.relu(self.conv1(x)))  
    x = self.pool(F.relu(self.conv2(x)))  
    temp = x.view(x.size()[0], -1)  
    x = self.dp1(x)  
    output = self.output(temp)  
    return output, x
```

构建基于CNN的猫狗分类网络

- 其次是搭建一个简单的VGG网络，包含8个卷积层、8个池化层、1个全连接层和1个丢弃层。
- 一般而言，VGG网络需要至少3个全连接层。
- 但是考虑到全连接层有大量的参数，为了减小电脑GPU的负荷，所以仅保留了一个全连接层。
- 并在每一个卷积层后都配置一个最大池化层，用于降低后续计算的参数量。
- 经过调整后的神经网络虽然在结构上发生了较大的变化，但是降低了运行代码所需的计算机资源。

构建基于CNN的猫狗分类网络

VGGNet

```
class MYVGG(nn.Module):
```

```
    def __init__(self, num_classes=2):
```

```
        super(MYVGG, self).__init__()
```

```
        self.conv1 = nn.Conv2d(3, 64, 3, padding=1)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.conv2 = nn.Conv2d(64, 64, 3, padding=1)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.conv4 = nn.Conv2d(128, 128, 3, padding=1)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
self.conv5 = nn.Conv2d(128, 256, 3, padding=1)
```

```
self.pool = nn.MaxPool2d(2, 2)
```

```
self.conv6 = nn.Conv2d(256, 256, 3, padding=1)
```

```
self.pool = nn.MaxPool2d(2, 2)
```

```
self.conv7 = nn.Conv2d(256, 512, 3, padding=1)
```

```
self.pool = nn.MaxPool2d(2, 2)
```

```
self.conv8 = nn.Conv2d(512, 512, 3, padding=1)
```

```
self.pool = nn.MaxPool2d(2, 2)
```

```
self.output = nn.Linear(512, num_classes)
```

```
self.dp1 = nn.Dropout(p=0.5)
```

构建基于CNN的猫狗分类网络

```
def forward(self, x):  
    x = self.pool(F.relu(self.conv1(x)))  
    x = self.pool(F.relu(self.conv2(x)))  
    x = self.pool(F.relu(self.conv3(x)))  
    x = self.pool(F.relu(self.conv4(x)))  
    x = self.pool(F.relu(self.conv5(x)))  
    x = self.pool(F.relu(self.conv6(x)))  
    x = self.pool(F.relu(self.conv7(x)))  
    x = self.pool(F.relu(self.conv8(x)))  
    temp = x.view(x.size()[0], -1)  
    x = self.dp1(x)  
    output = self.output(temp)  
    return output, x
```

构建基于CNN的猫狗分类网络

AlexNet

- 最后搭建一个简单的AlexNet网络，包含5个卷积层、5个池化层、1个全连接层和1个丢弃层。

```
class AlexNet(nn.Module):
```

```
    def __init__(self):
```

```
        super(AlexNet,self).__init__()
```

```
        self.conv1 = nn.Conv2d(3, 32, 3)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.conv2 = nn.Conv2d(32, 64, 3)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.conv3 = nn.Conv2d(64, 128, 3)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.conv4 = nn.Conv2d(128, 256, 3)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.conv5 = nn.Conv2d(256, 512, 3)
```

```
        self.pool = nn.MaxPool2d(2, 2)
```

```
        self.output = nn.Linear(in_features=512  
* 6 * 6, out_features=2)
```

```
        self.dp1 = nn.Dropout(p=0.5)
```

构建基于CNN的猫狗分类网络

```
def forward(self,x):
```

```
    x = self.pool(F.relu(self.conv1(x)))
```

```
    x = self.pool(F.relu(self.conv2(x)))
```

```
    x = self.pool(F.relu(self.conv3(x)))
```

```
    x = self.pool(F.relu(self.conv4(x)))
```

```
    x = self.pool(F.relu(self.conv5(x)))
```

```
    temp = x.view(x.shape[0], -1)
```

```
    x = self.dp1(x)
```

```
    output = self.output(temp)
```

```
    return output, x
```

```
model1 = AlexNet().to(device)
```

```
summary(model1, input_size=(3, 256, 256))
```

Layer (type)	Output Shape	Param #
=====	=====	=====
Conv2d-1	[-1, 32, 254, 254]	896
MaxPool2d-2	[-1, 32, 127, 127]	0
Conv2d-3	[-1, 64, 125, 125]	18,496
MaxPool2d-4	[-1, 64, 62, 62]	0
Conv2d-5	[-1, 128, 60, 60]	73,856
MaxPool2d-6	[-1, 128, 30, 30]	0
Conv2d-7	[-1, 256, 28, 28]	295,168
MaxPool2d-8	[-1, 256, 14, 14]	0
Conv2d-9	[-1, 512, 12, 12]	1,180,160
MaxPool2d-10	[-1, 512, 6, 6]	0
Dropout-11	[-1, 512, 6, 6]	0
Linear-12	[-1, 2]	36,866

=====

Total params: 1,605,442

Trainable params: 1,605,442

Non-trainable params: 0

Input size (MB): 0.75

Forward/backward pass size (MB): 36.35

Params size (MB): 6.12
